UNIVERSIDADE DE MOGI DAS CRUZES
ENGENHARIA DE SOFTWARE

ANDRÉ QUINTINO AIRES AFONSO
CARLOS VINICIUS BORBA RIBEIRO
KAUET YUDI SUGAWARA BRITO

ESTEIRA DEVSECOPS2

Secrets Scanning, SCA e SAST aplicados ao projeto EmprestaCampus

Mogi das Cruzes
2026

ANDRÉ QUINTINO AIRES AFONSO
CARLOS VINICIUS BORBA RIBEIRO
KAUET YUDI SUGAWARA BRITO

ESTEIRA DEVSECOPS

Secrets Scanning, SCA e SAST aplicados ao projeto EmprestaCampus

Relatório técnico do projeto EmprestaCampus apresentado ao curso de Engenharia de Software da Universidade de Mogi das Cruzes, como requisito parcial de avaliação da atividade DevOps 2.

Mogi das Cruzes
2026

SUMÁRIO

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO 4
2 DESCRIÇÃO DAS FERRAMENTAS ADOTADAS 5
2.1 Secrets scanning: TruffleHog 5
2.2 SCA: Trivy 5
2.3 SAST: Semgrep OSS 6
2.4 Política de bloqueio 6
2.5 Proteção da branch principal 7
2.6 Endurecimento da própria esteira 7
3 INTEGRAÇÃO COM O DOCUMENTO DE REQUISITOS 9
4 EVIDÊNCIAS DE FUNCIONAMENTO 11
4.1 Do primeiro workflow às execuções verdes 11
4.2 Bloqueios reais e mitigação 12
4.3 Bloqueio de Pull Request 13
4.4 Alertas no painel de segurança 17
4.5 Release da entrega 19
5 CONCLUSÃO 20

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO

Em um modelo tradicional, a segurança costuma ser verificada no fim do desenvolvimento, quando corrigir uma falha é mais caro e mais demorado. A abordagem SecDevOps (ou DevSecOps) desloca essas verificações para dentro do próprio fluxo de integração: cada alteração enviada ao repositório é analisada automaticamente, e o código com falhas graves é barrado antes de chegar à branch principal. Esse princípio, conhecido como *shift-left*, é recomendado pelo *Secure Software Development Framework* do NIST (SP 800-218).

A automação traz três ganhos em relação à revisão manual. Primeiro, a verificação acontece **em todas as alterações**, sem depender da memória ou da disponibilidade de alguém. Segundo, o critério é **sempre o mesmo**, definido em arquivo versionado. Terceiro, cada execução deixa **registro**: quem alterou, quando e com qual resultado, o que apoia auditoria e prestação de contas.

O EmprestaCampus é um sistema web de empréstimo de equipamentos acadêmicos, desenvolvido em Django, Django REST Framework e PostgreSQL. Ele trata autenticação com 2FA, recuperação de senha, dados pessoais protegidos pela LGPD, dados criptografados em repouso com Fernet (AES) e trilha de auditoria. Por isso, a equipe integrou ao repositório, no GitHub Actions, o workflow `.github/workflows/devsecops-pipeline.yml`, com três verificações executadas a cada push e Pull Request para a branch `main`: varredura de segredos (TruffleHog), análise de vulnerabilidades em dependências (Trivy) e análise estática de segurança (Semgrep). A esteira bloqueia o build quando encontra falhas de severidade alta ou crítica, e uma regra de proteção da branch impede o merge enquanto as verificações não forem aprovadas.

2 DESCRIÇÃO DAS FERRAMENTAS ADOTADAS

As três ferramentas são gratuitas e de código aberto, e rodam no próprio runner do GitHub Actions. Cada uma cobre uma vertente diferente de risco, como resume o Quadro 1.

Quadro 1 – Ferramentas adotadas por vertente

Vertente	Ferramenta	O que analisa	Job no workflow
Secrets scanning	TruffleHog 3.99.8	Histórico completo do Git	`Secrets Scanning - TruffleHog`
SCA	Trivy 0.76.1	Imagem Docker do backend (Python e Debian)	`SCA - Trivy`
SAST	Semgrep OSS 1.184.0	Código-fonte, templates, Dockerfile e YAML	`SAST - Semgrep`

Fonte: elaborado pelos autores (2026).

2.1 Secrets scanning: TruffleHog

O TruffleHog procura senhas, chaves de API, tokens e strings de conexão em **todos os commits** do repositório, e não apenas nos arquivos atuais. Isso é essencial porque uma credencial removida de um arquivo continua acessível no histórico do Git. O diferencial técnico que motivou a escolha é a **verificação**: para muitos tipos de credencial, a ferramenta testa se ela ainda é válida e classifica o achado como verificado (*verified*) ou não verificado (*unknown*). Com isso, foi possível definir um bloqueio rigoroso sem travar a esteira por credenciais antigas que já não funcionam.

O job usa duas etapas bloqueantes: a primeira varre o histórico inteiro e falha se houver qualquer segredo **verificado**; a segunda varre apenas os **commits novos** do push ou do Pull Request e falha com qualquer segredo, verificado ou não. Assim, nenhuma credencial nova entra no repositório. Um relatório JSON do histórico completo é publicado como artefato.

2.2 SCA: Trivy

A análise de composição de software (SCA) verifica se as bibliotecas de terceiros têm vulnerabilidades conhecidas (CVEs). O Trivy foi escolhido por consultar várias bases públicas (NVD, GitHub Security Advisories e o rastreador de segurança do Debian) e por analisar tanto pacotes de linguagem quanto pacotes do sistema operacional.

A equipe optou por analisar a **imagem Docker construída**, e não apenas o repositório. A imagem contém exatamente o que vai para produção: as versões instaladas das bibliotecas Python, suas dependências transitivas e os pacotes do Debian da imagem base `python:3.14-slim`. Essa decisão se mostrou importante: o Trivy encontrou vulnerabilidades em bibliotecas embutidas no próprio `pip`, que uma análise só do `requirements.txt` não enxergaria (Seção 4.2).

2.3 SAST: Semgrep OSS

A análise estática (SAST) examina o código-fonte em busca de padrões inseguros, sem executar a aplicação. O Semgrep foi escolhido por três motivos: funciona em repositórios públicos e privados; tem regras específicas para Django, Django REST Framework, templates HTML, Dockerfile e YAML (o conjunto `p/default` aplicou 142 regras ao projeto); e gera relatórios no formato SARIF, que o GitHub exibe nativamente em *Security* > *Code scanning*.

O enunciado permite o GitHub CodeQL como alternativa para repositórios públicos. A equipe manteve o Semgrep porque ele também atenderia a um repositório privado, cobre arquivos que o CodeQL não analisa (templates Django e Dockerfile) e, com o envio do SARIF, os achados aparecem no mesmo painel de segurança usado pelo CodeQL.

2.4 Política de bloqueio

A atividade exige que a esteira bloqueie o build quando encontrar falhas de severidade alta ou crítica. O Quadro 2 mostra o critério de cada job e o que é apenas registrado, sem bloquear.

Quadro 2 – Critério de bloqueio de cada job

Job	Bloqueia (exit-code diferente de 0) quando	Registra sem bloquear
Secrets Scanning - TruffleHog	Segredo verificado em qualquer commit do histórico; ou qualquer segredo, mesmo não verificado, nos commits novos. Código de saída 183.	Todos os achados do histórico, no artefato `trufflehog-relatorio`.
SCA - Trivy	CVE de severidade HIGH ou CRITICAL que já tenha versão corrigida disponível.	Todas as CVEs, inclusive sem correção, no artefato `trivy-sca-relatorio`.
SAST - Semgrep	Achado de severidade ERROR (alta).	Achados WARNING e INFO, enviados ao Code Scanning e ao artefato `semgrep-relatorio`.

Fonte: elaborado pelos autores (2026).

As CVEs sem correção publicada (comuns em pacotes do Debian) não bloqueiam porque não haveria ação possível: a esteira ficaria vermelha indefinidamente. Elas continuam no relatório completo e são reavaliadas a cada execução. O Quadro 3 mostra os trechos do workflow que implementam o bloqueio.

Quadro 3 – Trechos do workflow que implementam o bloqueio

```
# TruffleHog - histórico completo (só segredos verificados)
trufflehog git file://. --results=verified --fail --no-update --github-actions

# TruffleHog - commits novos (verificados e não verificados)
trufflehog git file://. --since-commit "$BASE" --branch "$HEAD" \
  --results=verified,unknown --fail --no-update --github-actions

# Trivy - CVEs HIGH/CRITICAL com correção disponível
trivy image emprestacampus-backend:${{ github.sha }} --scanners vuln \
  --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1

# Semgrep - achados de severidade ERROR
semgrep scan --config p/default --metrics=off --severity ERROR --error .
```

Fonte: arquivo .github/workflows/devsecops-pipeline.yml (2026).

2.5 Proteção da branch principal

Um job reprovado, sozinho, não impede o merge: ele apenas marca a execução como falha. Para que a esteira de fato impeça a integração de código defeituoso, foi criado no GitHub o *ruleset* **Devops2**, aplicado à branch `main`, com as seguintes regras:

- **Require a pull request before merging:** nenhuma alteração entra por push direto; tudo passa por Pull Request.
- **Require status checks to pass:** os jobs `Secrets Scanning - TruffleHog`, `SCA - Trivy` e `SAST - Semgrep` são obrigatórios; enquanto um deles não for aprovado, o botão de merge fica desabilitado.
- **Block force pushes** e **Restrict deletions:** o histórico da `main` não pode ser reescrito nem apagado.
- **Lista de exceções (bypass) vazia:** a regra vale para todos, inclusive para o dono do repositório.

2.6 Endurecimento da própria esteira

Uma esteira de segurança também faz parte da cadeia de suprimentos do software e pode ser atacada. Por isso, a equipe aplicou ao próprio workflow os controles a seguir.

- **Actions fixadas pelo SHA do commit**, e não por tag. Em 19 de março de 2026, um atacante reescreveu as tags de versão do `aquasecurity/trivy-action` para distribuir código que roubava segredos de CI/CD (CVE-2026-33634). Workflows que referenciavam a tag executaram o código malicioso; os fixados por SHA não foram afetados.
- **Binários verificados por SHA-256.** O TruffleHog e o Trivy são baixados do GitHub Releases e só são executados se o hash conferir com o arquivo oficial de checksums.
- **Versões fixas** das ferramentas, inclusive da imagem do Semgrep (`semgrep/semgrep:1.184.0`), para que o resultado não mude sozinho entre execuções.
- **Permissões mínimas:** o workflow só tem permissão de leitura (`contents: read`); apenas o job do Semgrep recebe `security-events: write`, necessária para enviar o SARIF ao painel de segurança.

3 INTEGRAÇÃO COM O DOCUMENTO DE REQUISITOS

O documento de requisitos do projeto (Requisitos 1.pdf, disponível em `docs/Requisitos.pdf`) organiza os critérios de segurança em oito grupos, de 1 (Autenticação e Gestão de Credenciais) a 8 (Pôster e Apresentação). A esteira não substitui a implementação desses requisitos: ela verifica, a cada alteração, que os controles já implementados não sejam enfraquecidos, e gera evidências para auditoria. O Quadro 4 relaciona os requisitos aos controles da esteira.

Quadro 4 – Relação entre os requisitos do projeto e a esteira automatizada

Requisito	Descrição no documento	Controle da esteira	Contribuição para auditoria, conformidade e proteção contínua
1.1 a 1.4	Hash seguro de senhas (PBKDF2), parâmetros de custo, salt e armazenamento	Semgrep	Regras para Django detectam hash fraco ou senha sem validação. O Semgrep apontou que a API não aplicava a política de senhas (CWE-521, Seção 4.4).
1.11	Proteção contra força bruta (rate limit, bloqueio)	Semgrep	Alerta `missing-throttle-config`: a API REST não tem limite de requisições, além do bloqueio de login do django-axes (Seção 4.4).
2.1 a 2.5	Recuperação de senha e token seguro	Semgrep	Achado bloqueante na `RedefinirSenhaView` (valor padrão vazio para a nova senha), corrigido antes do merge (Seção 4.2).
3.1 a 3.3	TLS/HTTPS e bloqueio de conexões inseguras	Trivy	Monitora CVEs nas bibliotecas de rede e criptografia da imagem (OpenSSL, zlib, `cryptography`).
3.4 a 3.6	Dados criptografados em repouso (AES) e chaves protegidas	TruffleHog	Impede que a `SECRET_KEY`, a `ENCRYPTION_KEY` (chave AES/Fernet) ou senhas do banco sejam versionadas. Um segredo novo bloqueia o PR (Seção 4.3).
4.1 a 4.11	Conformidade com a LGPD	As três	Atende ao art. 46 (medidas técnicas de segurança) e ao art. 6º, VIII (prevenção) e X (prestação de contas): cada versão tem relatórios datados e reproduzíveis.
5.1 a 5.3	Logs e proteção contra alteração	Ruleset e histórico do Actions	O ruleset impede reescrever ou apagar o histórico da `main` (force push e exclusão), e cada execução fica registrada com autor, data e resultado.
6.7 e 6.8	Ameaças, vulnerabilidades e associação risco × contramedida	As três	Cada achado é associado a uma mitigação (Quadros 6 e 7).
6.9 e 6.10	Testes de segurança realizados e resultados documentados	As três	A esteira executa testes de segurança automaticamente e publica os resultados como artefatos de cada execução.
6.11	Uso de normas técnicas	Todas	Critérios alinhados ao NIST SP 800-218, à classificação CWE e ao OWASP Top 10.

Fonte: elaborado pelos autores (2026).

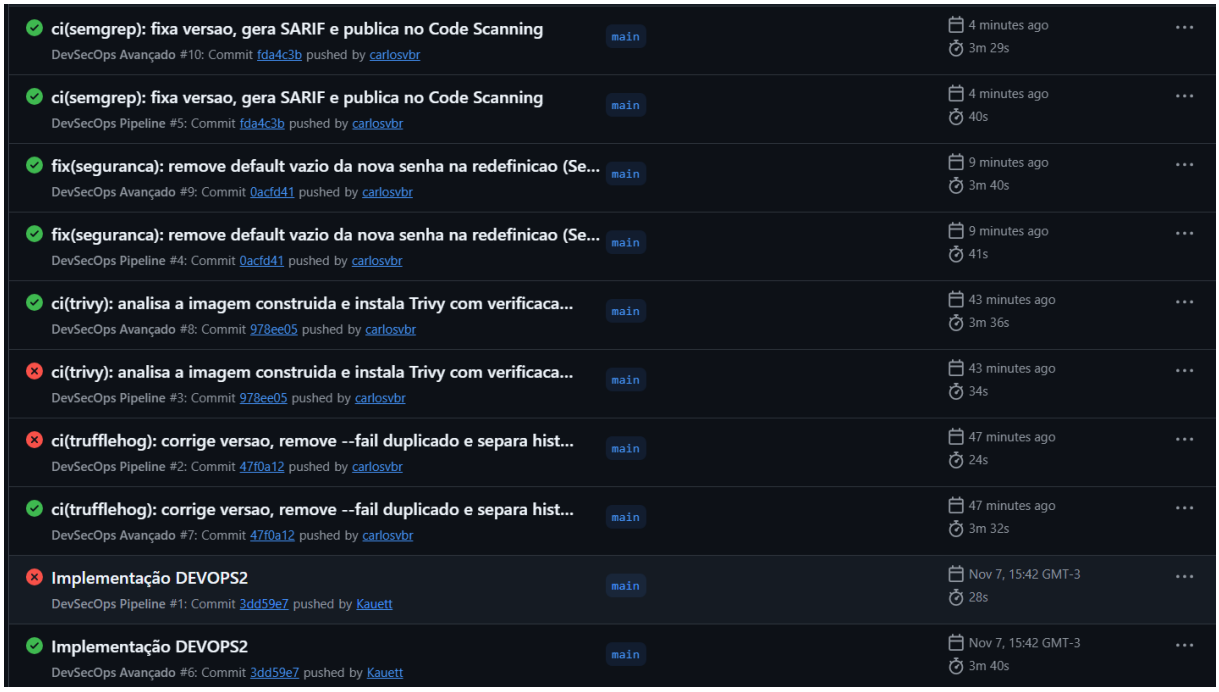
A integração tem limites que a equipe considera importante registrar. Os scanners apontam padrões e componentes vulneráveis, mas não provam que a criptografia foi usada corretamente, nem substituem os testes funcionais de autenticação, autorização por papel e fluxos da LGPD. Um resultado verde significa que os critérios definidos foram executados e aprovados, e não que o sistema está livre de vulnerabilidades.

4 EVIDÊNCIAS DE FUNCIONAMENTO

4.1 Do primeiro workflow às execuções verdes

A primeira versão do workflow não funcionava, e as correções ficaram registradas no histórico de execuções (Figura 1). O Quadro 5 resume a sequência.

Figura 1 – Histórico de execuções: falhas iniciais e execuções aprovadas após as correções



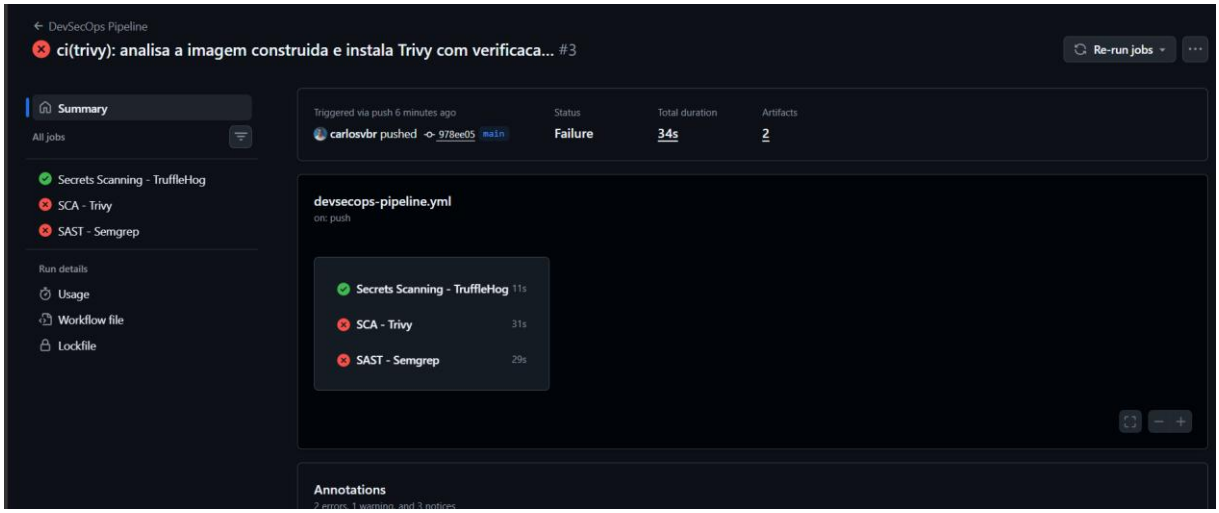
Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

Quadro 5 – Sequência de execuções do DevSecOps Pipeline

Execução	Resultado	O que aconteceu
#1 (`3dd59e7`)	Reprovada	Erro de configuração: a referência `trufflesecurity/trufflehog@v3` não existe e o parâmetro `--fail` estava duplicado. O SCA analisava o repositório com o `requirements.txt` sem versões fixas e, por isso, não identificava nenhuma dependência.
#2 (`47f0a12`)	Reprovada	TruffleHog corrigido e aprovado; Trivy e Semgrep ainda na versão inicial.
#3 (`978ee05`)	Reprovada	Trivy e Semgrep **bloquearam falhas reais** (Figuras 2 e 3; Seção 4.2).
#4 (`0acfd41`) e #5 (`fda4c3b`)	Aprovadas	Após as mitigações, os três jobs passaram.

Fonte: elaborado pelos autores (2026).

Figura 2 – Execução #3: TruffleHog aprovado; Trivy e Semgrep reprovados



Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

Figura 3 – Anotações da execução #3: Trivy e Semgrep encerrados com exit code 1



Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

4.2 Bloqueios reais e mitigação

Na execução #3, o Trivy encontrou sete vulnerabilidades de severidade HIGH com correção disponível e reprovou o job. O Quadro 6 lista os achados e a mitigação aplicada.

Quadro 6 – Vulnerabilidades bloqueadas pelo Trivy e mitigação

Pacote	CVE	Origem	Mitigação
zlib1g 1.3.1 (Debian)	CVE-2026-85091 (heap overflow em `gz_vacate()`) e CVE-2026-76844	Imagem base anterior à correção do Debian	Atualização dos pacotes do sistema no Dockerfile (`apt-get upgrade`)
msgpack 1.1.2	CVE-2026-114613 e GHSA-6v7p-g79w-8964	Cópia embutida no pip 26.2.2	Remoção do pip da imagem final
setuptools 70.3.0	CVE-2025-47273	Cópia embutida no pip	Remoção do pip da imagem final

Pacote	CVE	Origem	Mitigação
urllib3 2.7.0	CVE-2026-97687 e CVE-2026-97689	Cópia embutida no pip	Remoção do pip da imagem final

Fonte: log do job SCA - Trivy, execução #3 (2026).

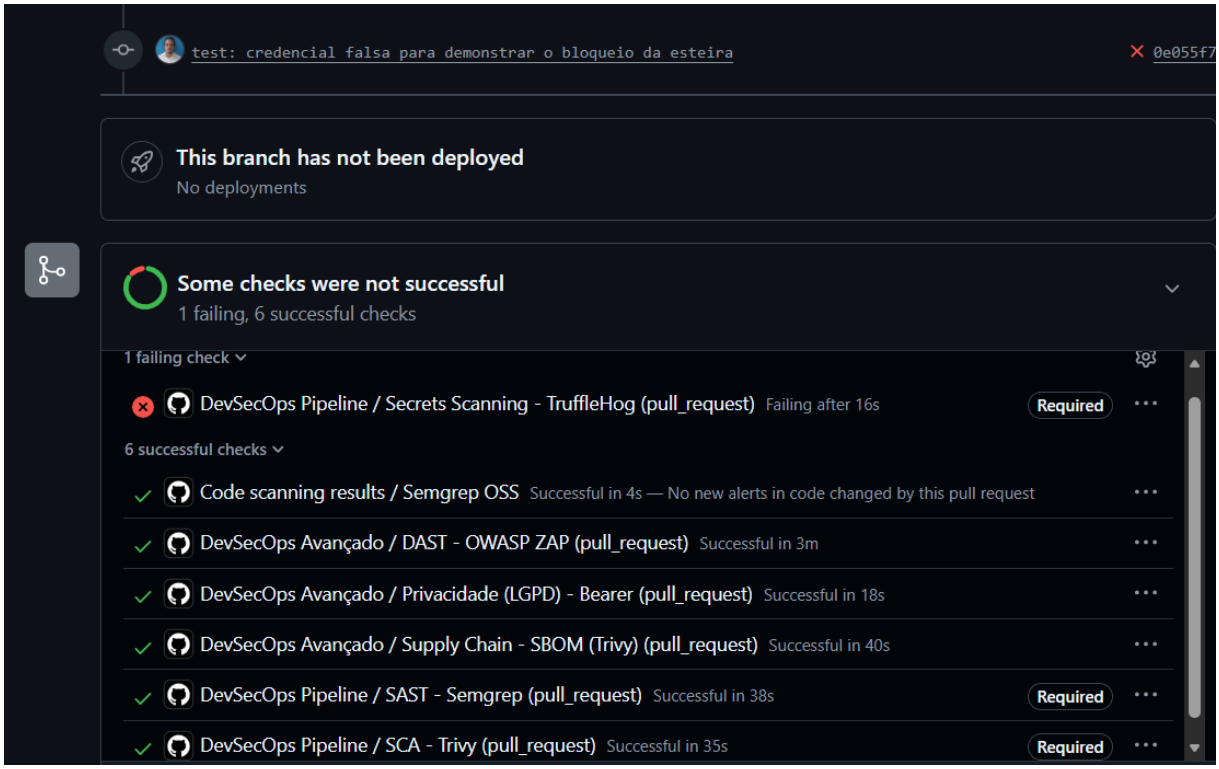
Nenhuma dependência direta do sistema (Django, Django REST Framework, cryptography, simplejwt, django-axes e django-otp) tinha vulnerabilidade. Cinco das sete CVEs estavam em bibliotecas que o próprio `pip` carrega consigo. Como o `pip` só é usado durante o build, ele foi removido da imagem final, seguindo o princípio da minimização: menos software instalado, menor superfície de ataque. A equipe também fixou as versões de todas as dependências no `requirements.txt`, tornando o build reproduzível.

Na mesma execução, o Semgrep reprovou o job com um achado de severidade ERROR: a regra `use-none-for-password-default` (CWE-521) apontou que a redefinição de senha, em `backend/usuarios/views.py`, usava uma string vazia como valor padrão da nova senha. A correção (commit `0acfd41`) passou a usar `None` como padrão e a validar o tipo do campo, o que também evita um erro 500 quando o campo é enviado como número. Na execução seguinte, os três jobs foram aprovados.

4.3 Bloqueio de Pull Request

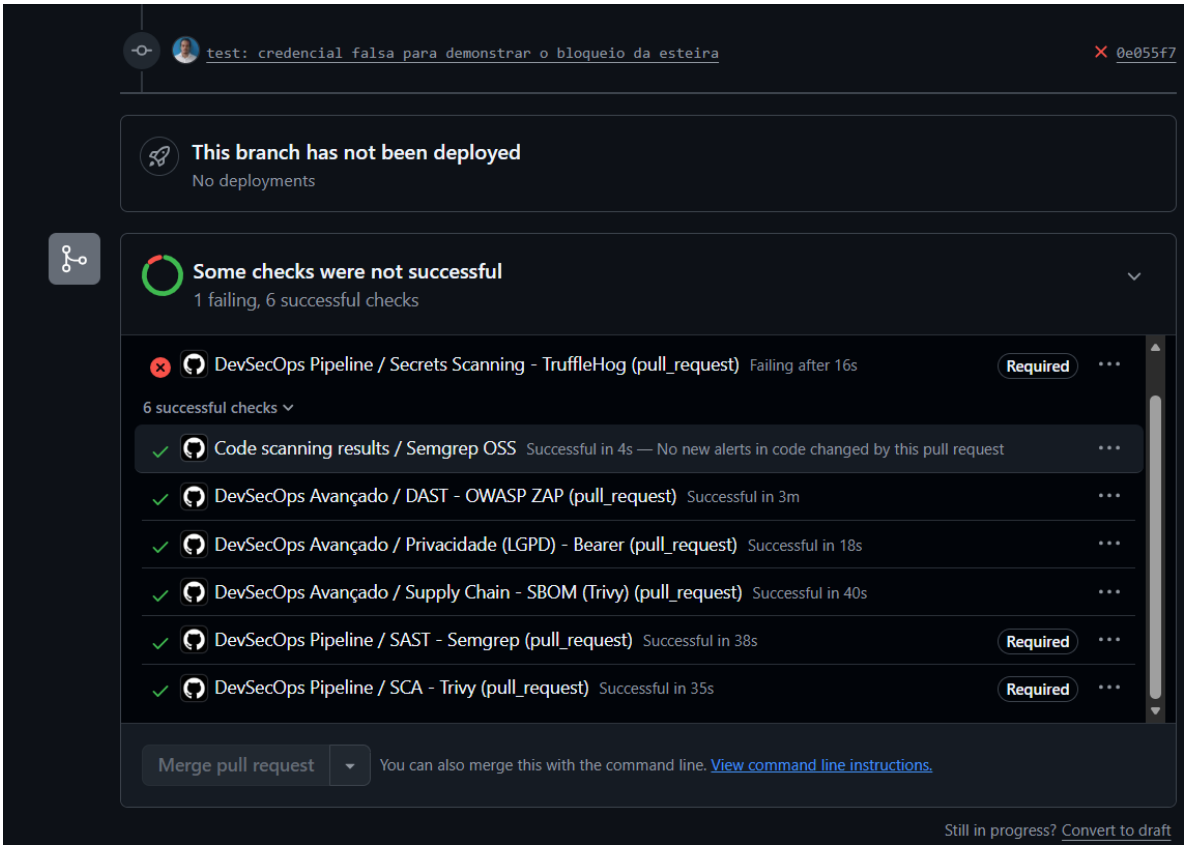
Para demonstrar que a esteira impede o merge, a equipe abriu um Pull Request com uma credencial falsa de banco de dados (`postgres://admin:...@db-producao.emprestacampus.local`). O check obrigatório do TruffleHog falhou (Figura 4) e o botão de merge ficou desabilitado (Figura 5). O log do job mostra a detecção e o código de saída 183 (Figura 6). O Pull Request foi fechado sem merge. A regra de proteção que produziu esse bloqueio aparece nas Figuras 7 a 9.

Figura 4 – Pull Request de teste: check obrigatório do TruffleHog reprovado



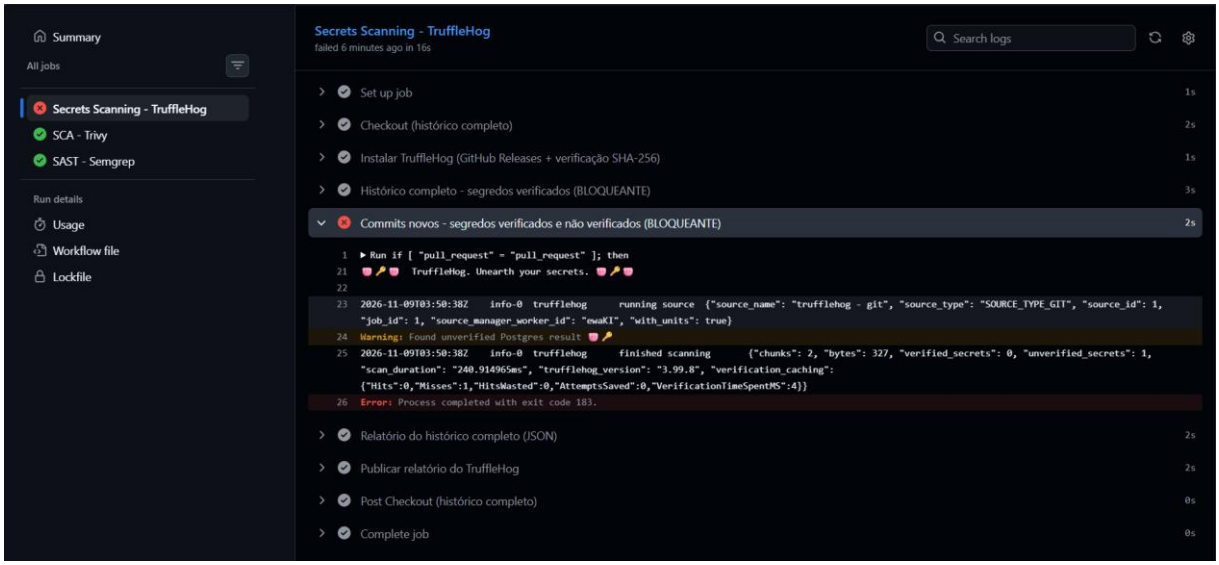
Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

Figura 5 – Pull Request de teste: botão de merge desabilitado



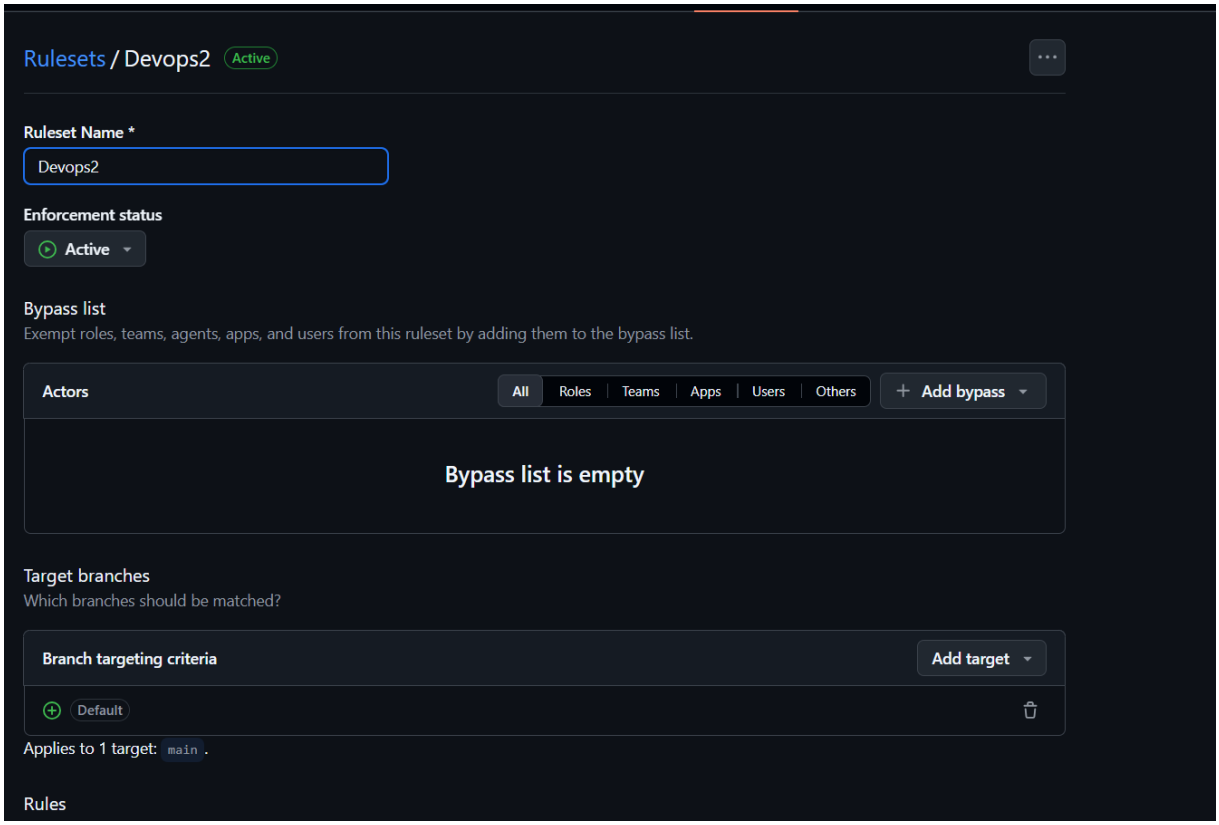
Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

Figura 6 – Log do TruffleHog: credencial não verificada detectada e exit code 183



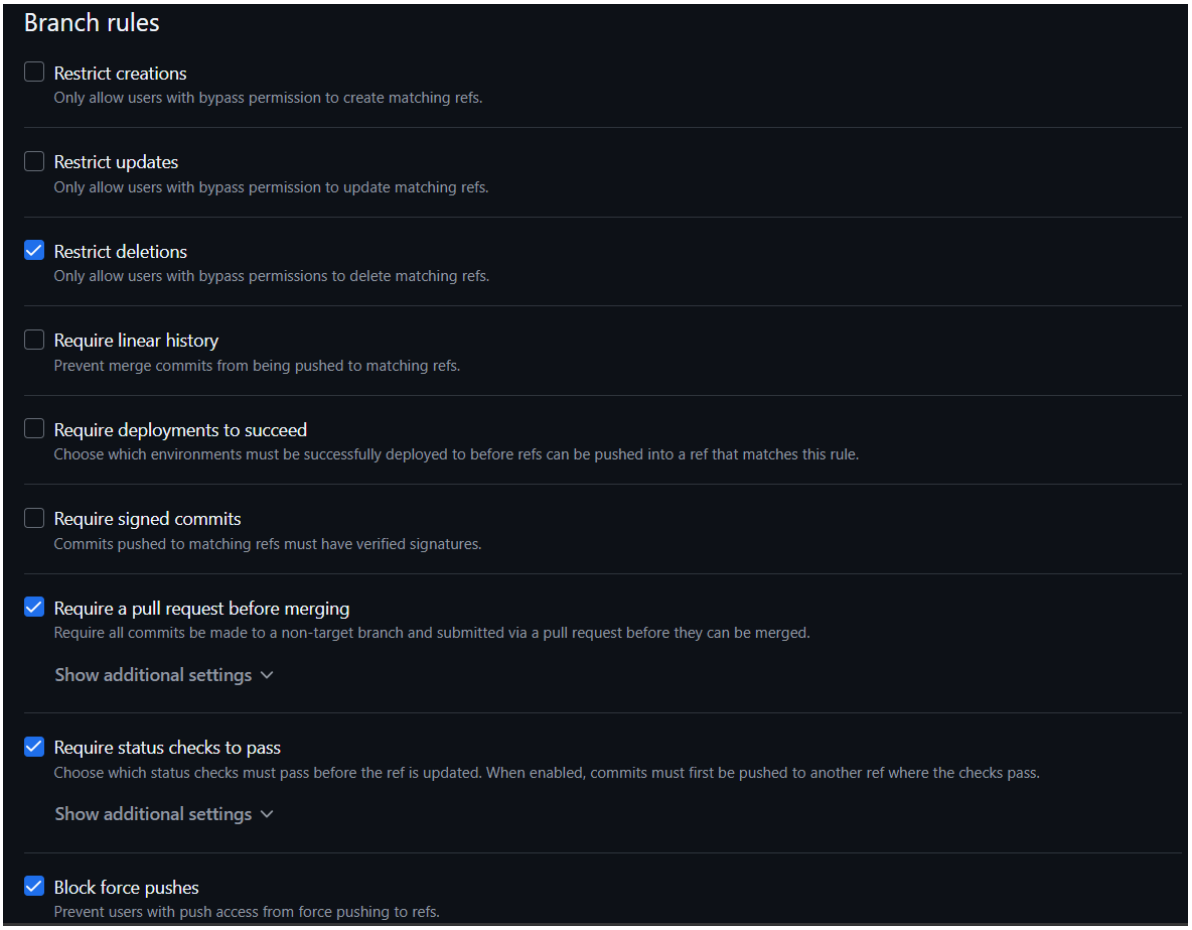
Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

Figura 7 – Ruleset Devops2 ativo, sem exceções, aplicado à branch main



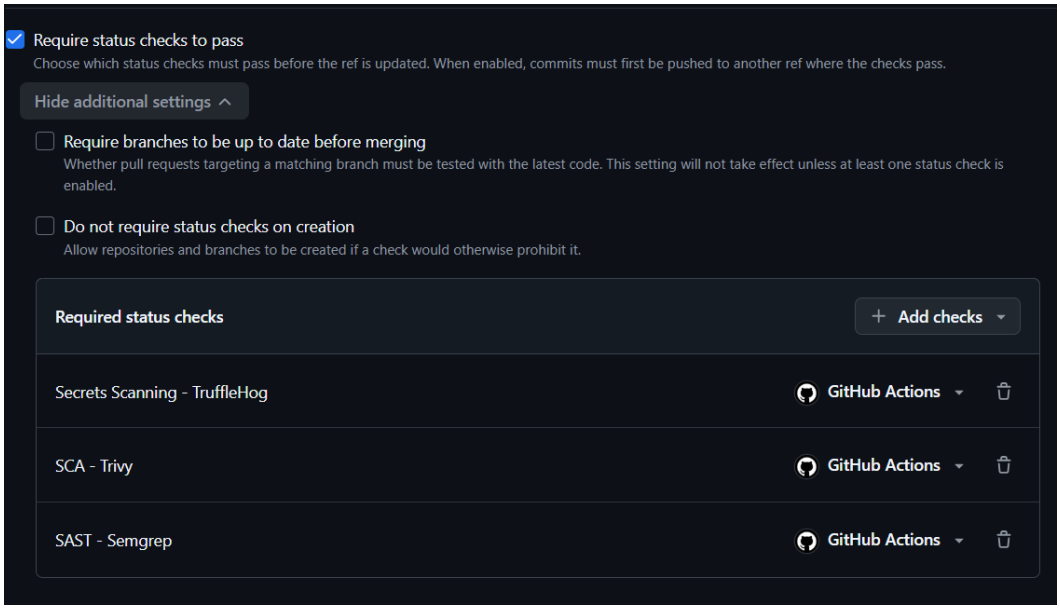
Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

Figura 8 – Regras do ruleset: Pull Request obrigatório, checks obrigatórios, bloqueio de force push e de exclusão



Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

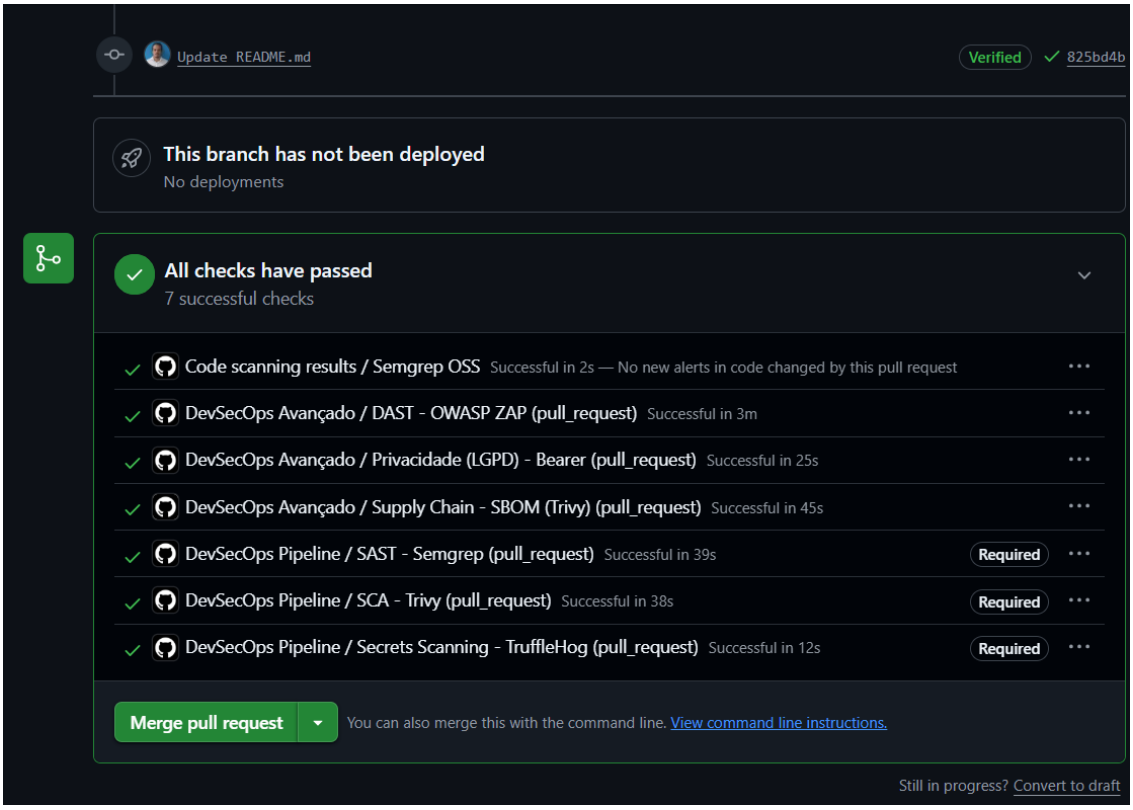
Figura 9 – Checks obrigatórios do ruleset: TruffleHog, Trivy e Semgrep



Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

Como contraponto, o Pull Request #43, que atualizou a documentação do README, passou nos três checks obrigatórios e teve o merge liberado (Figura 10).

Figura 10 – Pull Request #43: checks aprovados e merge liberado



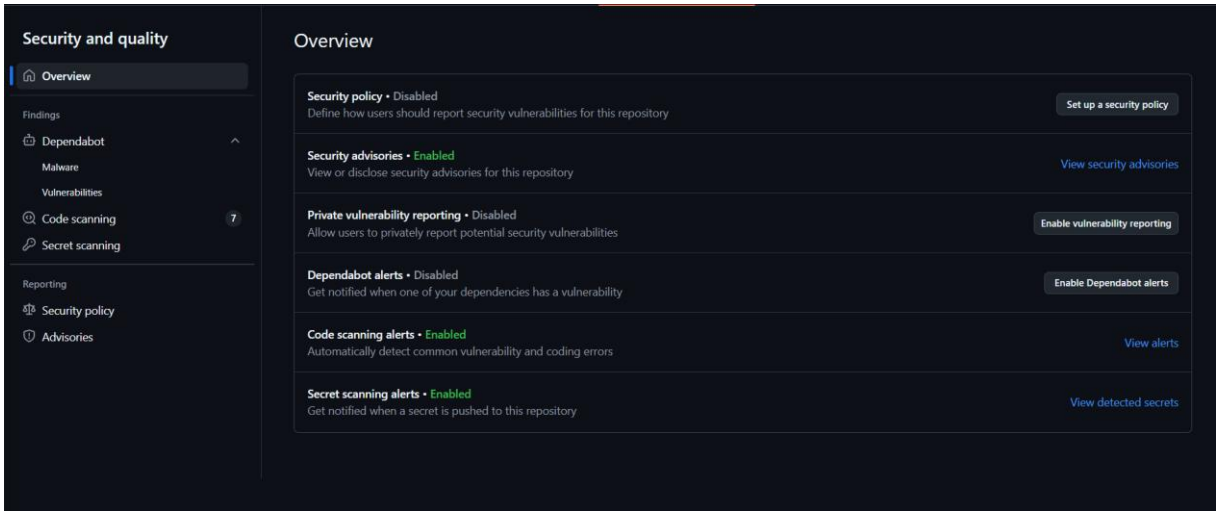
Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

A varredura do histórico completo também registrou uma credencial não verificada: a senha padrão de desenvolvimento do PostgreSQL (`postgres:postgres`), presente no `docker-compose.yml` no commit `c2fb036`, de 13/08/2026. Ela já foi removida do arquivo atual, que passou a usar `env_file`, e nunca foi usada fora do ambiente local. Reescrever o histórico do Git foi descartado por ser uma operação destrutiva e desnecessária para uma senha sem validade. Por isso, o histórico bloqueia apenas segredos verificados, enquanto os commits novos bloqueiam qualquer segredo.

4.4 Alertas no painel de segurança

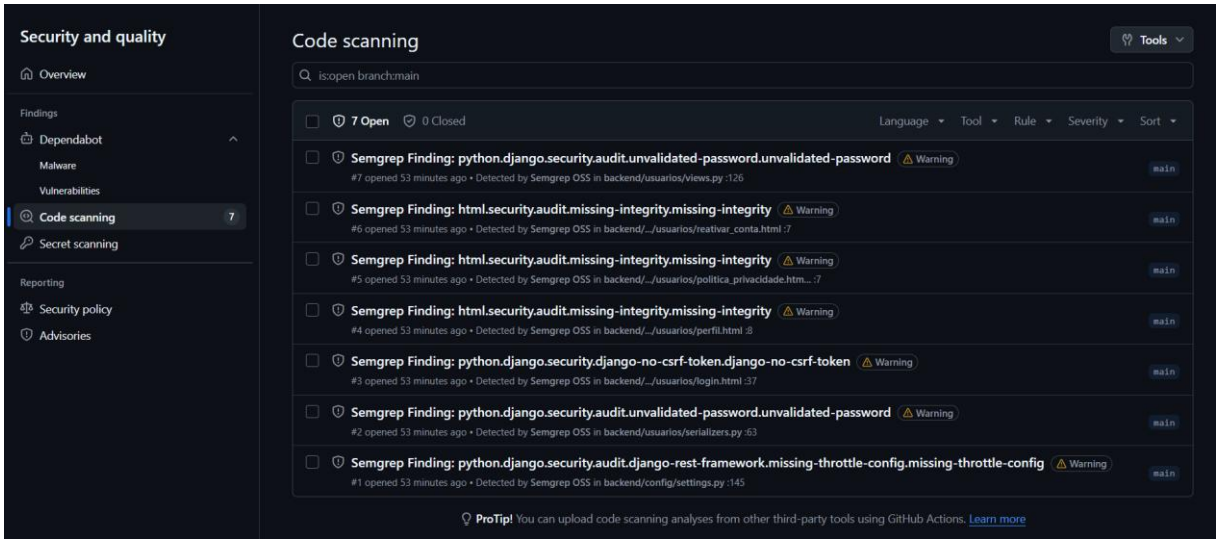
O relatório SARIF do Semgrep é enviado ao painel *Security* > *Code scanning* do GitHub (Figura 11), que registrou sete alertas de severidade WARNING (Figura 12). Esses alertas não bloqueiam a esteira, mas foram analisados pela equipe. O Quadro 7 descreve como cada um seria mitigado.

Figura 11 – Painel Security and quality: Code scanning e Secret scanning ativos



Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

Figura 12 – Code scanning: sete alertas do Semgrep na branch main



Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

Quadro 7 – Alertas do Code Scanning e mitigação

Nº	Regra	Local	Análise e mitigação
#7	`unvalidated-password`	`usuarios/views.py:126` (redefinição de senha)	**Falha real.** O `settings.py` define quatro validadores de senha, mas a API só exige 8 caracteres e aceita senhas como `12345678`. Mitigação: chamar `validate_password()` antes de `set_password()`.
#2	`unvalidated-password`	`usuarios/serializers.py:63` (cadastro)	**Falha real.** Mesmo problema no cadastro. Mitigação: aplicar `validate_password()` no método `create()` do serializer.
#1	`missing-throttle-config`	`config/settings.py:145`	**Risco parcial.** O login é protegido pelo django-axes, mas o restante da API não tem limite de requisições (CWE-770). Mitigação: configurar `DEFAULT_THROTTLE_CLASSES` e `DEFAULT_THROTTLE_RATES` do DRF.

Nº	Regra	Local	Análise e mitigação
#3 a #5	`missing-integrity`	`perfil.html`, `politica_privacidade.html` e `reativar_conta.html`	**Falha real.** O Tailwind é carregado de `cdn.tailwindcss.com` sem verificação de integridade (SRI); esse CDN gera o CSS dinamicamente e não aceita hash. Mitigação: compilar o CSS no build e servi-lo como arquivo local.
#6	`django-no-csrf-token`	`usuarios/login.html:37`	**Falso positivo.** O formulário tem `{% csrf_token %}` na linha seguinte; a regra roda em modo genérico e não interpreta o template. Mitigação: dispensar o alerta no GitHub com justificativa.

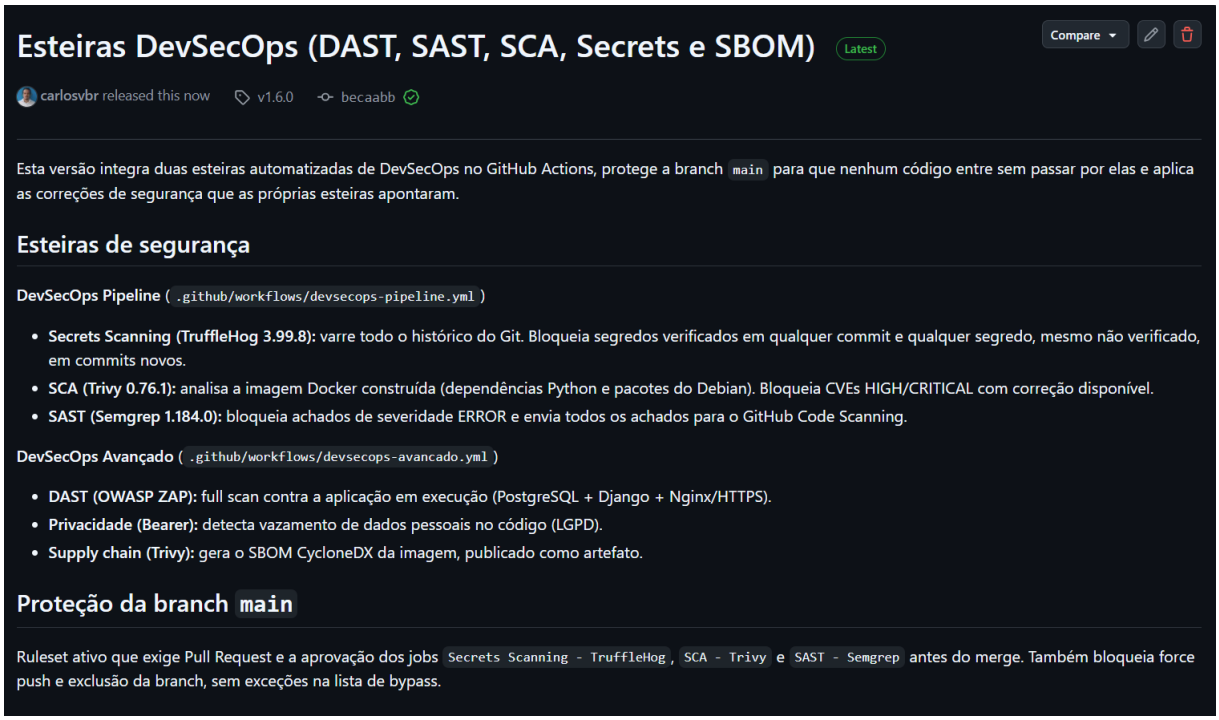
Fonte: GitHub Code Scanning, repositório carlosvbr/EmprestaCampus (2026).

As correções dos alertas #1, #2 e #7 foram implementadas e testadas pela equipe, com a suíte de 149 testes automatizados aprovada, e estão previstas para a próxima versão.

4.5 Release da entrega

A versão `v1.6.0` foi publicada em 09/11/2026, às 01:14, a partir do commit `becaabb` da branch `main`, que já inclui a esteira, as correções e a documentação no README (Figura 13).

Figura 13 – Release v1.6.0 publicada no GitHub



Fonte: GitHub, repositório carlosvbr/EmprestaCampus (2026).

5 CONCLUSÃO

A esteira DevSecOps do EmprestaCampus executa, a cada push e Pull Request, as três verificações solicitadas: varredura de segredos com TruffleHog, análise de dependências com Trivy e análise estática com Semgrep. A política de bloqueio foi comprovada com falhas reais: o Trivy barrou sete vulnerabilidades de severidade alta e o Semgrep barrou uma fragilidade na redefinição de senha. Ambas foram corrigidas antes de chegar à branch principal. O ruleset da `main` transforma esses resultados em bloqueio efetivo do merge, como demonstrado pelo Pull Request com credencial falsa.

A equipe identifica como próximos passos: aplicar as correções dos alertas #1, #2 e #7; compilar o CSS do Tailwind localmente para eliminar a dependência de CDN sem verificação de integridade; separar as dependências de teste (pytest) da imagem de produção; e manter as versões das ferramentas e das actions atualizadas, sempre fixadas por hash.